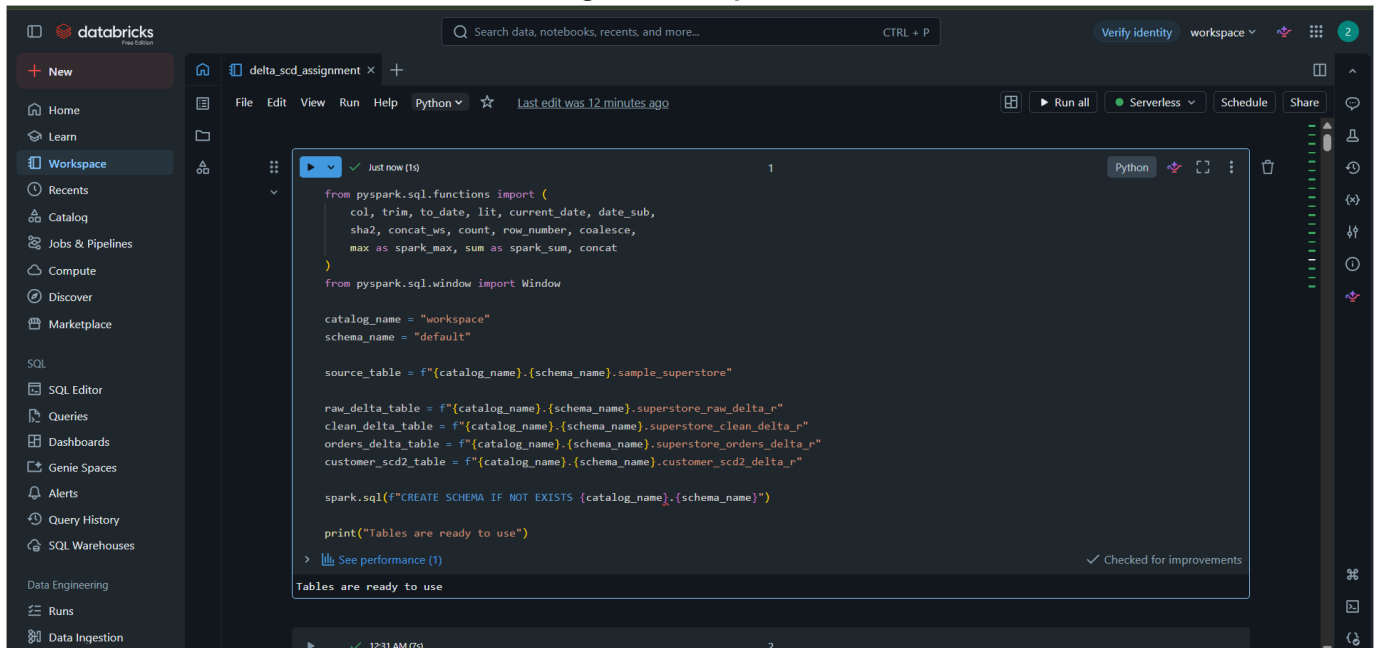


Delta Lake Incremental Processing Assignment

Objective: Perform incremental data processing using Delta Lake. The screenshots show dataset loading, cleaning, incremental data creation, MERGE operation, result validation, final output, and SCD Type 2 implementation.

Figure 1: Setup Code



```
from pyspark.sql.functions import (
    col, trim, to_date, lit, current_date, date_sub,
    sha2, concat_ws, count, row_number, coalesce,
    max as spark_max, sum as spark_sum, concat
)
from pyspark.sql.window import Window

catalog_name = "workspace"
schema_name = "default"

source_table = f"{catalog_name}.{schema_name}.sample_superstore"

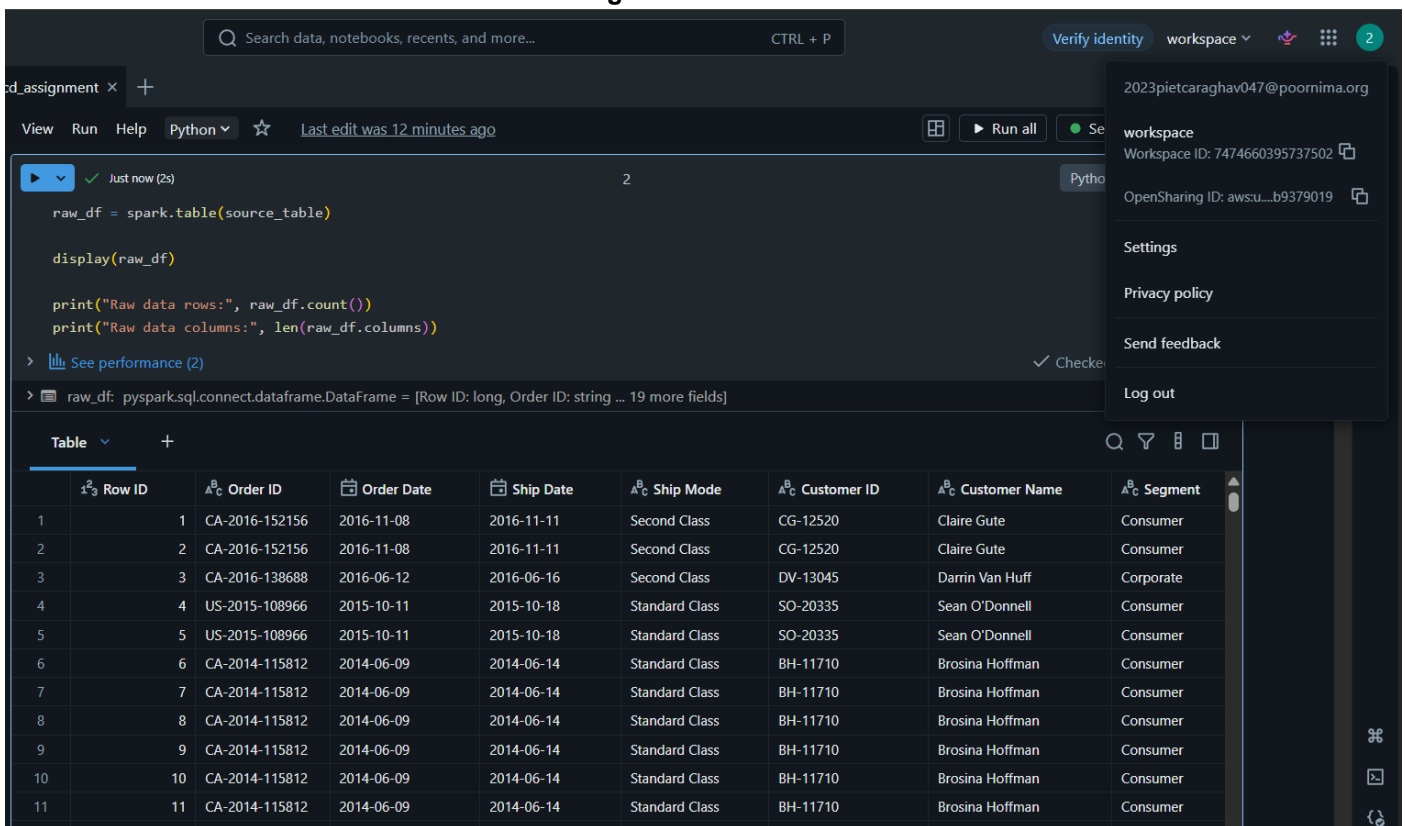
raw_delta_table = f"{catalog_name}.{schema_name}.superstore_raw_delta_r"
clean_delta_table = f"{catalog_name}.{schema_name}.superstore_clean_delta_r"
orders_delta_table = f"{catalog_name}.{schema_name}.superstore_orders_delta_r"
customer_scd2_table = f"{catalog_name}.{schema_name}.customer_scd2_delta_r"

spark.sql(f"CREATE SCHEMA IF NOT EXISTS {catalog_name}.{schema_name}")

print("Tables are ready to use")
```

Tables are ready to use

Figure 2: Raw Data



```
raw_df = spark.table(source_table)

display(raw_df)

print("Raw data rows:", raw_df.count())
print("Raw data columns:", len(raw_df.columns))
```

raw_df: pyspark.sql.connect.dataframe.DataFrame = [Row ID: long, Order ID: string ... 19 more fields]

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment
1	1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer
2	2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer
3	3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate
4	4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer
5	5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer
6	6	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
7	7	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
8	8	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
9	9	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
10	10	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
11	11	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer

Figure 3: Data Check

```
raw_missing_df = raw_df.select([
    spark_sum(col(c).isNull().cast("int")).alias(c)
    for c in raw_df.columns
])

display(raw_missing_df)

raw_duplicate_count = raw_df.count() - raw_df.dropDuplicates().count()

print("Duplicate rows found in raw data:", raw_duplicate_count)
```

raw_missing_df: pyspark.sql.connect.dataframe.DataFrame = [Row ID: long, Order ID: long ... 19 more fields]

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment
1	0	0	0	0	0	0	0

1 row | 3.097s runtime

Duplicate rows found in raw data: 0

Figure 4: Column Cleanup

```
working_df = raw_df.toDF("new_columns")

display(working_df.limit(10))
print("Column names are cleaned")
```

working_df: pyspark.sql.connect.dataframe.DataFrame = [row_id: long, order_id: string ... 19 more fields]

row_id	order_id	order_date	ship_date	ship_mode	customer_id	customer_name	segment
1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer
2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer
3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Darin Van Huff	Corporate
4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer
5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer
6	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
7	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
8	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
9	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
10	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer

10 rows | 1.361s runtime

Column names are cleaned

Figure 5: Incremental Logic

```
from pyspark.sql.functions import row_number, concat
from pyspark.sql.window import Window
from pyspark.sql.functions import max as spark_max

master_df = spark.table(target_table)

# existing records for update
existing_updates = (
    master_df
    .orderBy("row_id")
    .limit(5)
    .withColumn("sales", col("sales") + 100)
    .withColumn("profit", col("profit") + 20)
)

# new records for insert
max_row_id = master_df.agg(spark_max("row_id")).collect()[0][0]

new_records = (
    master_df
    .orderBy("row_id")
    .limit(3)
    .withColumn("rn", row_number().over(Window.orderBy("row_id")))
    .withColumn("row_id", lit(max_row_id) + col("rn"))
    .withColumn("order_id", concat(lit("NEW-ORDER-"), col("rn")))
    .withColumn("customer_id", concat(lit("NEW-CUST-"), col("rn")))
)
```

Figure 6: Incremental Output

```
incremental_df = existing_updates.unionByName(new_records)

display(incremental_df)

print("Incremental records:", incremental_df.count())
```

> [See performance \(3\)](#) ✓ Checked for improvements

> master_df: pyspark.sql.connect.dataframe.DataFrame = [row_id: integer, order_id: string ... 19 more fields]

> existing_updates: pyspark.sql.connect.dataframe.DataFrame = [row_id: integer, order_id: string ... 19 more fields]

> new_records: pyspark.sql.connect.dataframe.DataFrame = [row_id: integer, order_id: string ... 19 more fields]

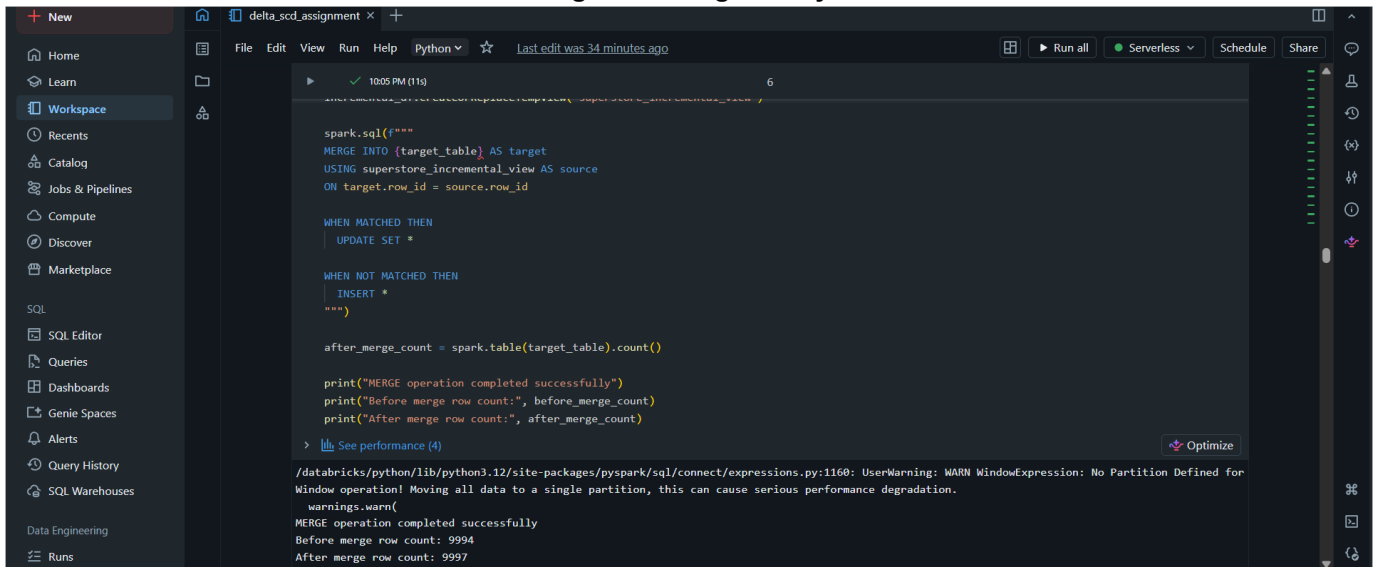
> incremental_df: pyspark.sql.connect.dataframe.DataFrame = [row_id: integer, order_id: string ... 19 more fields]

/databricks/python/lib/python3.12/site-packages/pyspark/sql/connect/expressions.py:1160: UserWarning: WARN WindowExpression: No Partition Defined for Window operation! Moving all data to a single partition, this can cause serious performance degradation.

warnings.warn(

	row_id	order_id	order_date	ship_date	ship_class	customer_id	customer_name	segment
1	1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer
2	2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer
3	3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate
4	4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'donnell	Consumer
5	5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'donnell	Consumer
6	9995	NEW-ORDER-1	2016-11-08	2016-11-11	Second Class	NEW-CUST-1	New Customer 1	Consumer

Figure 7: Merge Query



```
spark.sql(f"""
MERGE INTO {target_table} AS target
USING superstore_incremental_view AS source
ON target.row_id = source.row_id

WHEN MATCHED THEN
  UPDATE SET *

WHEN NOT MATCHED THEN
  INSERT *
""")

after_merge_count = spark.table(target_table).count()

print("MERGE operation completed successfully")
print("Before merge row count:", before_merge_count)
print("After merge row count:", after_merge_count)
```

> [See performance \(4\)](#) [Optimize](#)

/databricks/python/lib/python3.12/site-packages/pyspark/sql/connect/expressions.py:1160: UserWarning: WARN WindowExpression: No Partition Defined for Window operation! Moving all data to a single partition, this can cause serious performance degradation.

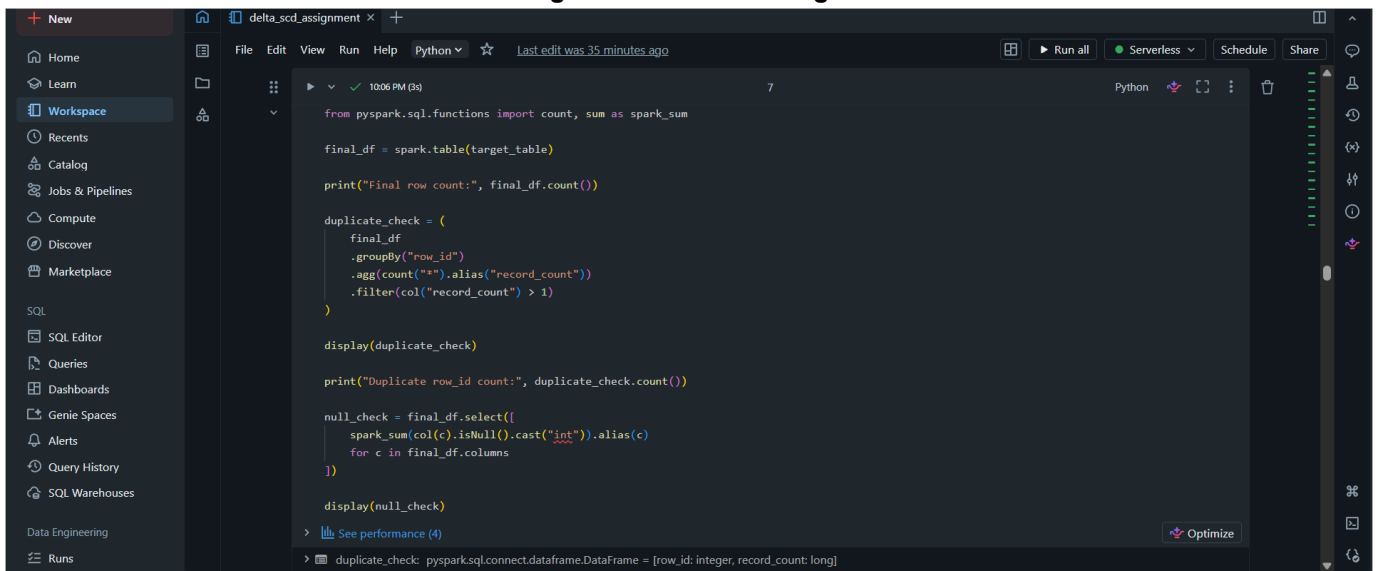
warnings.warn(

MERGE operation completed successfully

Before merge row count: 9994

After merge row count: 9997

Figure 8: Validation Logic



```
from pyspark.sql.functions import count, sum as spark_sum

final_df = spark.table(target_table)

print("Final row count:", final_df.count())

duplicate_check = (
    final_df
    .groupBy("row_id")
    .agg(count("*").alias("record_count"))
    .filter(col("record_count") > 1)
)

display(duplicate_check)

print("Duplicate row_id count:", duplicate_check.count())

null_check = final_df.select([
    spark_sum(col(c).isNull().cast("int")).alias(c)
    for c in final_df.columns
])

display(null_check)
```

> [See performance \(4\)](#) [Optimize](#)

> [duplicate_check: pyspark.sql.connect.dataframe.DataFrame = \[row_id: integer, record_count: long\]](#)

Figure 9: Duplicate Check

```
> duplicate_check: pyspark.sql.connect.dataframe.DataFrame = [row_id: integer, record_count: long]
> final_df: pyspark.sql.connect.dataframe.DataFrame = [row_id: integer, order_id: string ... 19 more fields]
> null_check: pyspark.sql.connect.dataframe.DataFrame = [row_id: long, order_id: long ... 19 more fields]
Final row count: 9997
```

Table

row_id	record_count
--------	--------------

No rows returned

0 rows | 3.447s runtime

Duplicate row_id count: 0

Table

row_id	order_id	order_date	ship_date	ship_mode	customer_id	customer_name	segment
1	0	0	0	0	0	0	0

Figure 10: Final Data

```
> display(final_df.orderBy("row_id"))
```

Table

row_id	order_id	order_date	ship_date	ship_mode	customer_id	customer_name	segment
2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer
3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Darin Van Huff	Corporate
4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'donnell	Consumer
5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'donnell	Consumer
6	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
7	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
8	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
9	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
10	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
11	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
12	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
13	CA-2017-114412	2017-04-15	2017-04-20	Standard Class	AA-10480	Andrew Allen	Consumer
14	CA-2016-161389	2016-12-05	2016-12-10	Standard Class	IM-15070	Irene Maddox	Consumer
15	US-2015-118983	2015-11-22	2015-11-26	Standard Class	HP-14815	Harold Pawlan	Home Office

Figure 11: Summary Logic

The screenshot shows the Databricks workspace interface. The left sidebar contains navigation options like Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Spaces, Alerts, Query History, SQL Warehouses, Data Engineering, and Runs. The main area displays a Python script for creating a summary DataFrame. The script defines a list of metrics and their values, then creates a DataFrame and displays it. The output is a table with 3 rows and 2 columns: Metric and Value.

```
summary_data = [
    ("Raw data rows", df_raw.count()),
    ("Cleaned data rows", df_clean.count()),
    ("Incremental data rows", incremental_df.count()),
    ("Final Delta table rows", final_df.count()),
    ("Duplicate row_id count", duplicate_check.count())
]

summary_df = spark.createDataFrame(summary_data, ["Metric", "Value"])

display(summary_df)
```

	Metric	Value
1	Raw data rows	9994
2	Cleaned data rows	9994
3	Incremental data rows	8

Figure 12: Summary Output

The screenshot shows the Databricks workspace interface. The left sidebar contains navigation options like Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Spaces, Alerts, Query History, SQL Warehouses, Data Engineering, and Runs. The main area displays the same Python script as in Figure 11. The output is a table with 5 rows and 2 columns: Metric and Value.

```
summary_data = [
    ("Duplicate row_id count", duplicate_check.count())
]

summary_df = spark.createDataFrame(summary_data, ["Metric", "Value"])

display(summary_df)
```

	Metric	Value
1	Raw data rows	9994
2	Cleaned data rows	9994
3	Incremental data rows	8
4	Final Delta table rows	9997
5	Duplicate row_id count	0

5 rows | 4.818s runtime

Refreshed 7 minutes ago

Figure 13: SCD2 Setup

The screenshot shows the Databricks SQL Editor interface. The left sidebar contains navigation options like Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Spaces, Alerts, Query History, SQL Warehouses, Data Engineering, and Runs. The main editor area displays a Python script for SCD2 setup. The script imports pyspark.sql.functions and defines source and target tables. It then creates a customer_master table by selecting base columns and dropping duplicates, and finally creates a customer_scd2 table by joining customer_master with a date range.

```
from pyspark.sql.functions import col, lit, to_date

source_table = "workspace.default.superstore_delta_assignment"
scd2_table = "workspace.default.customer_scd2_delta"

base_cols = [
    "customer_id",
    "customer_name",
    "segment",
    "country",
    "city",
    "state",
    "postal_code",
    "region"
]

customer_master = (
    spark.table(source_table)
    .select(base_cols)
    .dropDuplicates(["customer_id"])
)

customer_scd2 = (
    customer_master
    .withColumn("effective_start_date", lit("2026-06-01").cast("date"))
    .withColumn("effective_end_date", lit(None).cast("date"))
)
```

Figure 14: Initial SCD2

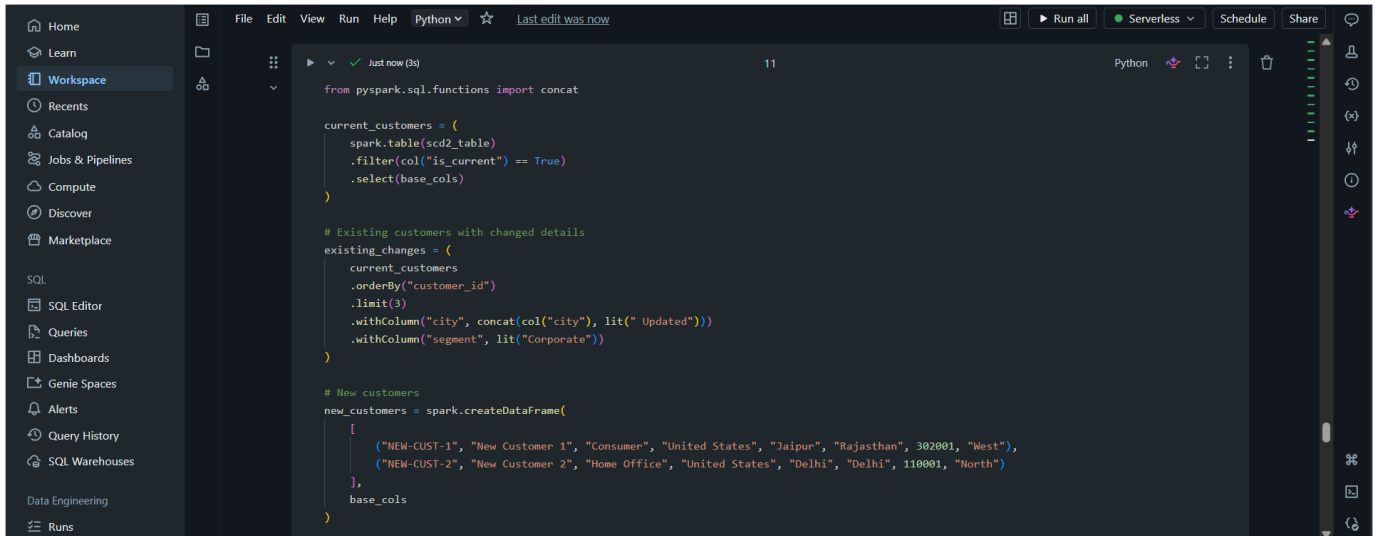
The screenshot shows the Databricks SQL Editor interface displaying the initial SCD2 table data. The table has 10 columns: customer_id, customer_name, segment, country, city, state, postal_code, region, and effective_start_date. The data is sorted by customer_id. Below the table, a message indicates that the SCD2 table was created successfully and the initial row count is 796.

	customer_id	customer_name	segment	country	city	state	postal_code	region	effective_start_date
1	AA-10315	Alex Avila	Consumer	United States	New York City	New York	10011	East	2026-06-01
2	AA-10375	Allen Arnold	Consumer	United States	Atlanta	Georgia	30318	South	2026-06-01
3	AA-10480	Andrew Allen	Consumer	United States	Springfield	Missouri	65807	Central	2026-06-01
4	AA-10645	Anna Andreadi	Consumer	United States	Georgetown	Kentucky	40324	South	2026-06-01
5	AB-10015	Aaron Bergman	Consumer	United States	Seattle	Washington	98103	West	2026-06-01
6	AB-10060	Adam Bellavance	Home Office	United States	Greenwood	Indiana	46142	Central	2026-06-01
7	AB-10105	Adrian Barton	Consumer	United States	Phoenix	Arizona	85023	West	2026-06-01
8	AB-10150	Aimee Bixby	Consumer	United States	Dallas	Texas	75220	Central	2026-06-01
9	AB-10165	Alan Barnes	Consumer	United States	Decatur	Illinois	62521	Central	2026-06-01
10	AB-10255	Alejandro Ballentine	Home Office	United States	Lorain	Ohio	44052	East	2026-06-01
11	AB-10600	Ann Blume	Corporate	United States	Tucson	Arizona	85705	West	2026-06-01
12	AC-10420	Alyssa Crouse	Corporate	United States	Philadelphia	Pennsylvania	19120	East	2026-06-01
13	AC-10450	Amy Cox	Consumer	United States	Avondale	Arizona	85323	West	2026-06-01
14	AC-10615	Ann Chong	Corporate	United States	Philadelphia	Pennsylvania	19143	East	2026-06-01

796 rows | 6.913s runtime

SCD2 table created successfully
Initial SCD2 row count: 796

Figure 15: Customer Changes



The screenshot shows a code editor with a dark theme. The left sidebar contains a navigation menu with options like Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Spaces, Alerts, Query History, SQL Warehouses, Data Engineering, and Runs. The main editor area displays Python code for handling customer data. The code includes imports, variable definitions for current and existing customers, and the creation of a new DataFrame for new customers.

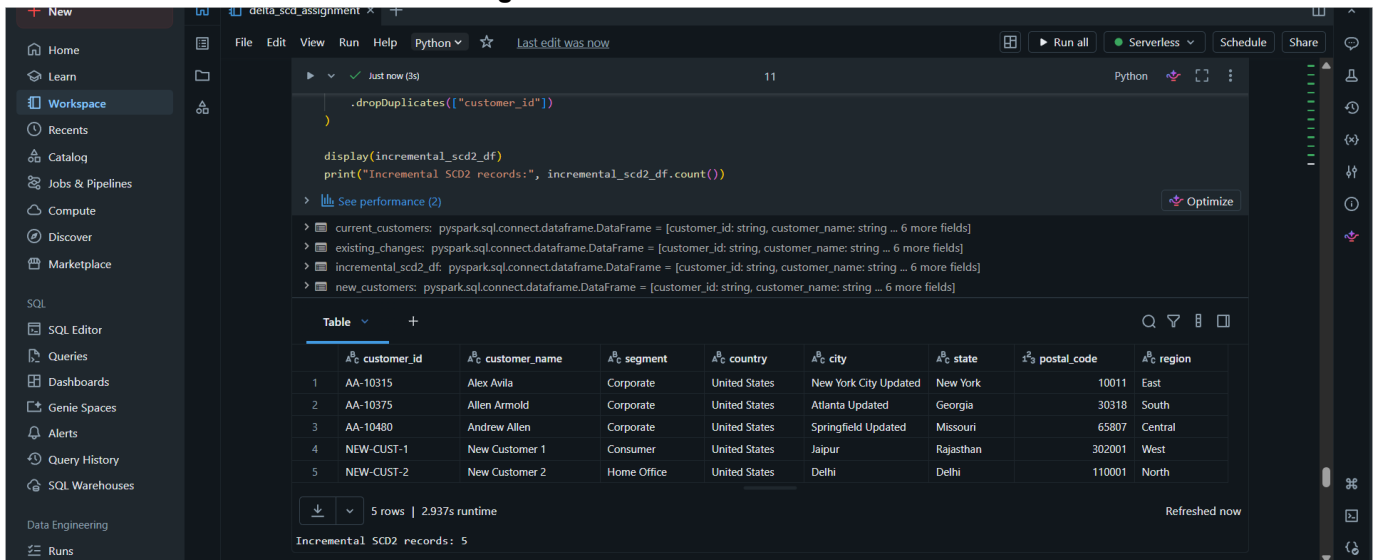
```
from pyspark.sql.functions import concat

current_customers = (
    spark.table("scd2_table")
    .filter(col("is_current") == True)
    .select(base_cols)
)

# Existing customers with changed details
existing_changes = (
    current_customers
    .orderBy("customer_id")
    .limit(3)
    .withColumn("city", concat(col("city"), lit(" Updated")))
    .withColumn("segment", lit("Corporate"))
)

# New customers
new_customers = spark.createDataFrame(
    [
        ("NEW-CUST-1", "New Customer 1", "Consumer", "United States", "Jaipur", "Rajasthan", 302001, "West"),
        ("NEW-CUST-2", "New Customer 2", "Home Office", "United States", "Delhi", "Delhi", 110001, "North")
    ],
    base_cols
)
```

Figure 16: Incremental Customers



The screenshot shows the same code editor as Figure 15, but with the code executed. The output pane at the bottom displays the results of the queries. It shows the current customers, existing changes, and new customers. A table view is also provided for the incremental SCD2 records.

```
.dropDuplicates(["customer_id"])

display(incremental_scd2_df)
print("Incremental SCD2 records:", incremental_scd2_df.count())
```

Output:

- current_customers: pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, customer_name: string ... 6 more fields]
- existing_changes: pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, customer_name: string ... 6 more fields]
- incremental_scd2_df: pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, customer_name: string ... 6 more fields]
- new_customers: pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, customer_name: string ... 6 more fields]

	customer_id	customer_name	segment	country	city	state	postal_code	region
1	AA-10315	Alex Avila	Corporate	United States	New York City Updated	New York	10011	East
2	AA-10375	Allen Arnold	Corporate	United States	Atlanta Updated	Georgia	30318	South
3	AA-10480	Andrew Allen	Corporate	United States	Springfield Updated	Missouri	65807	Central
4	NEW-CUST-1	New Customer 1	Consumer	United States	Jaipur	Rajasthan	302001	West
5	NEW-CUST-2	New Customer 2	Home Office	United States	Delhi	Delhi	110001	North

Incremental SCD2 records: 5

Figure 17: Hash Check

```

from pyspark.sql.functions import sha2, concat_ws, coalesce, current_date, max as spark_max

tracked_cols = [
    "customer_name",
    "segment",
    "country",
    "city",
    "state",
    "postal_code",
    "region"
]

def hash_columns(cols):
    return sha2(
        concat_ws("||", *[coalesce(col(c).cast("string"), lit("")) for c in cols]),
        256
    )

target_current = (
    spark.table(scd2_table)
    .filter(col("is_current") == True)
    .withColumn("target_hash", hash_columns(tracked_cols))
    .select("customer_id", "target_hash")
)

```

Figure 18: Insert Records

```

display(records_to_insert)
print("Records to insert in SCD2:", records_to_insert.count())

```

See performance (2) [Optimize](#)

latest_version: pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, latest_version: integer]
 records_to_insert: pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, customer_name: string ... 10 more fields]
 source_hashed: pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, customer_name: string ... 8 more fields]
 target_current: pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, target_hash: string]

	customer_id	customer_name	segment	country	city	state	postal_code	region
1	AA-10315	Alex Avila	Corporate	United States	New York City Updated	New York	10011	East
2	AA-10375	Allen Arnold	Corporate	United States	Atlanta Updated	Georgia	30318	South
3	AA-10480	Andrew Allen	Corporate	United States	Springfield Updated	Missouri	65807	Central
4	NEW-CUST-1	New Customer 1	Consumer	United States	Jaipur	Rajasthan	302001	West
5								

5 rows | 4.570s runtime Refreshed now

Records to insert in SCD2: 5

Figure 19: Expire Records

The screenshot shows a Databricks workspace interface. The left sidebar contains navigation options like Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Spaces, Alerts, Query History, SQL Warehouses, Data Engineering, and Runs. The main area displays a SQL query in the editor, titled "delta_scd_assignment". The query is a MERGE statement that updates records in a Delta table based on a source view. The query is as follows:

```
records_to_insert.createOrReplaceTempView("scd2_changes_view")

spark.sql(f"""
MERGE INTO {scd2_table} AS target
USING scd2_changes_view AS source
ON target.customer_id = source.customer_id
AND target.is_current = true

WHEN MATCHED THEN
  UPDATE SET
    target.is_current = false,
    target.effective_end_date = date_sub(source.effective_start_date, 1)
""")

print("Old current records expired successfully")
```

The query has been executed successfully, as indicated by the "Just now (6s)" status and the "Old current records expired successfully" message at the bottom. The interface also shows a "Run all" button, a "Serverless" dropdown, and a "Schedule" button. The top bar indicates the current session is "Last edit was now" and shows the date "2026-07-06".

Figure 20: New Versions

The screenshot shows a Databricks workspace interface. The left sidebar contains navigation options like Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Spaces, Alerts, Query History, SQL Warehouses, Data Engineering, and Runs. The main area displays a Python script in the editor, titled "delta_scd_assignment". The script is as follows:

```
records_to_insert_typed = records_to_insert.withColumn("postal_code", col("postal_code").cast("int"))

records_to_insert_typed.write.format("delta").mode("append").saveAsTable(scd2_table)

print("New SCD2 version records inserted successfully")
```

The script has been executed successfully, as indicated by the "Just now (4s)" status and the "New SCD2 version records inserted successfully" message at the bottom. The interface also shows a "Run all" button, a "Serverless" dropdown, and a "Schedule" button. The top bar indicates the current session is "Last edit was now" and shows the date "2026-07-06".

Figure 21: Final SCD2

New SCD2 version records inserted successfully

```

scd2_final = spark.table(scd2_table)

display(scd2_final.orderBy("customer_id", "version"))

print("Final SCD2 row count:", scd2_final.count())
print("Current active records:", scd2_final.filter(col("is_current") == True).count())
print("Historical inactive records:", scd2_final.filter(col("is_current") == False).count())

```

See performance (4) ✓ Checked for improvements

scd2_final: pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, customer_name: string ... 10 more fields]

	customer_id	customer_name	segment	country	city	state	postal_code	region
1	AA-10315	Alex Avila	Consumer	United States	New York City	New York	10011	East
2	AA-10375	Allen Arnold	Consumer	United States	Atlanta	Georgia	30318	South
3	AA-10480	Andrew Allen	Consumer	United States	Springfield	Missouri	65807	Central
4	AA-10645	Anna Andreadi	Consumer	United States	Georgetown	Kentucky	40324	South
5	AA-10645	Anna Andreadi	Corporate	United States	Georgetown Updat...	Kentucky	40324	South
6	AB-10015	Aaron Bergman	Consumer	United States	Seattle	Washington	98103	West
7	AB-10015	Aaron Bergman	Corporate	United States	Seattle Updated	Washington	98103	West

Figure 22: SCD2 Count

	customer_id	customer_name	segment	country	city	state	postal_code	region
30	AG-10900	Arthur Gainer	Consumer	United States	Waterbury	Connecticut	6708	East
31	AH-10030	Aaron Hawkins	Corporate	United States	San Francisco	California	94122	West
32	AH-10075	Adam Hart	Corporate	United States	Columbus	Ohio	43229	East
33	AH-10120	Adrian Hane	Home Office	United States	Louisville	Kentucky	40214	South
34	AH-10195	Alan Haines	Corporate	United States	New York City	New York	10024	East
35	AH-10210	Alan Hwang	Consumer	United States	Seattle	Washington	98105	West
36	AH-10465	Amy Hunt	Consumer	United States	New York City	New York	10035	East
37	AH-10585	Angele Hood	Consumer	United States	Seattle	Washington	98105	West
38	AH-10690	Anna H Berlin	Corporate	United States	Plantation	Florida	33317	South
39	AI-10855	Arianne Irving	Consumer	United States	Los Angeles	California	90036	West
40	AJ-10780	Anthony Jacobs	Corporate	United States	Scottsdale	Arizona	85254	West
41	AJ-10795	Anthony Johnson	Corporate	United States	San Francisco	California	94110	West
42	AJ-10945	Ashley Jarboe	Consumer	United States	Detroit	Michigan	48227	Central
43	AJ-10960	Astrea Jones	Consumer	United States	New York City	New York	10035	East

801 rows | 3.351s runtime Refreshed now

Final SCD2 row count: 801
Current active records: 796
Historical inactive records: 5

Figure 23: Current Validation



Figure 24: Duplicate Result

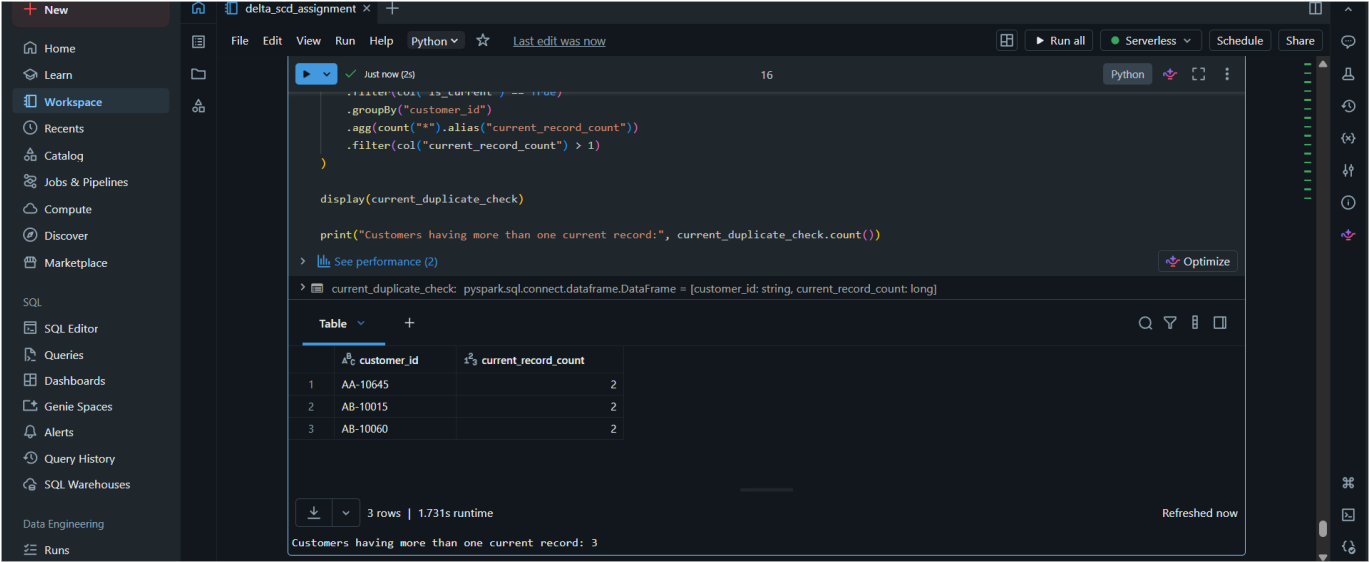


Figure 25: History Records

The screenshot shows a Databricks workspace with a Python notebook titled 'delta_scd_assignment'. The notebook contains a `display()` command that filters and orders data. Below the code, a table of history records is displayed, showing columns for customer_id, customer_name, segment, country, city, state, postal_code, and region. The table contains 5 rows of data.

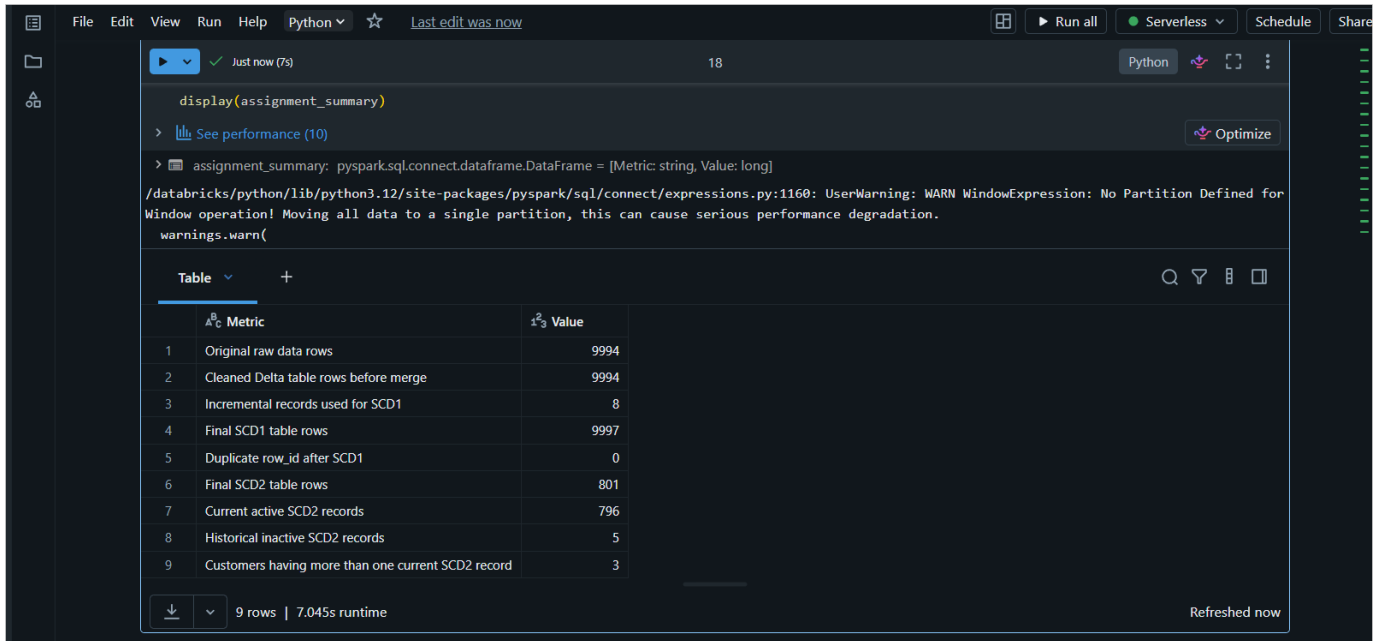
	customer_id	customer_name	segment	country	city	state	postal_code	region
1	AA-10645	Anna Andreadi	Corporate	United States	Georgetown Updat...	Kentucky	40324	South
2	AB-10015	Aaron Bergman	Corporate	United States	Seattle Updated	Washington	98103	West
3	AB-10060	Adam Bellavance	Corporate	United States	Greenwood Updated	Indiana	46142	Central
4	NEW-CUST-1	New Customer 1	Consumer	United States	Jaipur	Rajasthan	302001	West

Figure 26: Final Summary

The screenshot shows a Databricks workspace with a Python notebook titled 'delta_scd_assignment'. The notebook contains a `summary_data` list and a `display()` command. Below the code, a table of summary data is displayed, showing columns for Metric and Value. The table contains 10 rows of data.

Metric	Value
Original raw data rows	df_raw.count()
Cleaned Delta table rows before merge	df_clean.count()
Incremental records used for SCD1	incremental_df.count()
Final SCD1 table rows	final_df.count()
Duplicate row_id after SCD1	duplicate_check.count()
Final SCD2 table rows	scd2_final.count()
Current active SCD2 records	scd2_final.filter(col("is_current") == True).count()
Historical inactive SCD2 records	scd2_final.filter(col("is_current") == False).count()
Customers having more than one current SCD2 record	current_duplicate_check.count()

Figure 27: Summary Output



The screenshot shows a Databricks notebook interface. At the top, there's a menu bar with 'File', 'Edit', 'View', 'Run', 'Help', 'Python', and a star icon. Below the menu, there's a toolbar with 'Run all', 'Serverless', 'Schedule', and 'Share' buttons. The main area displays a code cell with the following content:

```
display(assignment_summary)
```

Below the code, there's a link 'See performance (10)' and an 'Optimize' button. A warning message is displayed:

```
/databricks/python/lib/python3.12/site-packages/pyspark/sql/connect/expressions.py:1160: UserWarning: WARN WindowExpression: No Partition Defined for Window operation! Moving all data to a single partition, this can cause serious performance degradation.
warnings.warn()
```

Below the warning, a table is shown with the following data:

	Metric	Value
1	Original raw data rows	9994
2	Cleaned Delta table rows before merge	9994
3	Incremental records used for SCD1	8
4	Final SCD1 table rows	9997
5	Duplicate row_id after SCD1	0
6	Final SCD2 table rows	801
7	Current active SCD2 records	796
8	Historical inactive SCD2 records	5
9	Customers having more than one current SCD2 record	3

At the bottom of the table, there's a download icon, a dropdown menu, and the text '9 rows | 7.045s runtime'. On the right side, there's a 'Refreshed now' button.

Short explanation

In this assignment, I loaded the Superstore dataset in Databricks and cleaned it by handling null values, removing duplicates, and fixing column names. After cleaning, saved the data as a Delta table. Then I created incremental records and used the Delta Lake MERGE command to update existing rows and insert new rows. Also validated the final output using row count and duplicate checks. SCD Type 2 was also added to maintain historical customer records.